

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Computer Science & Engineering

Assignment No. : 01

Assignment Title : Number Theory And Modular Arithmetic

Course Code : CSE 2101

Course Title : Discrete Mathematics

<u>Submitted by:</u> Sanzidul Islam Siam Roll: 2303074 Section: B Session: 2023-24	<u>Submitted to:</u> Utsha Das Assistant Professor Dept. of Computer Science & Engineering, RUET
---	---

Date of Submission : 22 February 2026

Question 1: Access Code Validation

The system requires that a valid access code must be divisible by both 4 and 6.

a) Explain how the divisibility of an integer is defined mathematically.

Solution:

If a and b are integers, then a divides b if $a * n = b$ for some integer n . In this case, a is a factor or a divisor of b . The notation $a \mid b$ means "a divides b".

Example:

16 divides 64 because if $n = 4$ then,

$$16 * 4 = 64$$

here 4 is an integer.

b) Write a Python program to check whether a given access code satisfies the above condition.

Solution:

Code:

```
def main():
    a = int(input("Give me the code: "))
    if(a%4==0 and a%6==0): print("Given access code is valid")
    else: print("Given access code is invalid")
if __name__=='__main__':
    main()
```

Output:

```
Give me the code: 48
Given access code is valid
```

C) Test your program using the access code C = 72.

Solution:

```
• Give me the code: 72
  Given access code is valid
```

Question 2: Security Cycle Synchronisation

Two security modules refresh their encryption keys periodically. One module refreshes every 36 minutes, while the other refreshes every 60 minutes.

a) Compute the Greatest Common Divisor (GCD) of the two time intervals.

Solution:

Given,

$$A = 36 = 2 * 18 = 2 * 2 * 9 = 2 * 2 * 3 * 3$$

$$B = 60 = 2 * 30 = 2 * 2 * 15 = 2 * 2 * 3 * 5$$

To calculate gcd, we take the common prime divisors. Which are 2, 2, 3

Hence, GCD = $2 * 2 * 3 = 12$. (ans)

b) Compute the Least Common Multiple (LCM) to determine after how many minutes both modules will refresh simultaneously.

Solution:

$$A = 36 = 2 * 18 = 2 * 2 * 9 = 2 * 2 * 3 * 3$$

$$B = 60 = 2 * 30 = 2 * 2 * 15 = 2 * 2 * 3 * 5$$

To calculate lcm, we take the common and uncommon prime divisors. Which are 2, 2, 3, 3, 5

Hence, LCM = $2 * 2 * 3 * 3 * 5 = 180$. (ans)

So after 180 minutes both modules will refresh simultaneously.

C) Implement the solution using Python.

Solution:

GCD and LCM implemented with python given below:

```
from math import gcd, lcm
def main():
    a, b = 36, 60
    print(f"The GCD: {gcd(a,b)}")
    print(f"The LCM: {lcm(a,b)}")
    print(f"After {lcm(a,b)} minutes both modules will refresh simultaneously.")
if __name__ == '__main__':
    main()
```

Output:

```
The GCD: 12
The LCM: 180
After 180 minutes both modules will refresh simultaneously.
```

Question 3: Secure Code Transformation

During transmission, an access code is transformed using modular arithmetic to reduce data size.

Given the access code $C = 17$ and modulus $n = 12$:

a) Compute $(C + 5) \bmod 12$.

Solution:

Given,

$$C = 17. \text{ Here, } c \bmod 12 = (12 * 1 + 5) \bmod 12 = 5$$

$$(c + 5) \bmod 12 = (17 + 5) \bmod 12 = (17 \bmod 12 + 5 \bmod 12) \bmod 12$$

$$= (5 + 5) \bmod 12 = 10 \bmod 12 = 10$$

Hence, $(C + 5) \bmod 12 = 10$. (ans)

b) Verify using Python that $(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$ for the given values.

Solution:

Let, for the given values, $a = 17$, $b = 5$, $n = 12$

Code:

```
from math import gcd, lcm
def check(a,b,n):
    return ((a+b)%n==(a%n+b%n)%n)
def main():
    a , b , n = 17, 5, 12
    print(f"The given statement for the given values is: {check(a,b,n)}")
if __name__=='__main__':
    main()
```

Output:

```
The given statement for the given values is: True
```

Question 4: Encryption Key Recovery

For encryption purposes, the system uses a key $a = 7$ under modulo 26.

a) Determine whether the modular inverse of 7 modulo 26 exists.

Solution:

The modular inverse of a number a modulo n exists only if $\gcd(a,n) = 1$

here, $a = 7$, $n = 26$

As $\gcd(7, 26) = 1$, hence, the modular inverse of 7 modulo 26 exists. (ans)

b) Compute the modular inverse using the Extended Euclidean Algorithm.

Solution:

To calculate modular inverse of 7 modulo 26

we write,

$$7 * x \equiv 1 \pmod{26}$$

where x is the modular inverse

now,

$$26 = 7 * 3 + 5 \dots\dots(1)$$

$$7 = 5 * 1 + 2 \dots\dots(2)$$

$$5 = 2 * 2 + 1 \dots\dots(3)$$

$$2 = 1 * 2 + 0 \dots\dots(4)$$

from eq (1),

$$5 = 26 - (7 * 3)$$

from eq (2),

$$2 = 7 - 5 * 1$$

$$\text{or, } 2 = 7 - (26 - 7 * 3) * 1$$

from eq (3),

$$1 = 5 - 2 * 2$$

$$\text{or, } 1 = 26 - 7 * 3 - (7 - (26 - 7 * 3)) * 2$$

or, $1 = 3 * 26 - 11 * 7$

or, $7 * -11 \equiv 1(mod 26)$

Here, $-11 \equiv 15(mod 26)$

Hence, $7^{-1} \equiv 15(mod 26)$

So, the modular inverse of 7 modulo 26 is 15. (ans)

c) Verify the result programmatically.

Solution:

Code:

```
def check(a,n,inv):
    return ((a*inv)%n==1)
def main():
    a , n, inv = 7, 26, 15
    print(f"The modular inverse of {a} modulo {n} is {inv}: {check(a,n, inv)}")
if __name__=='__main__':
    main()
```

Output:

```
The modular inverse of 7 modulo 26 is 15: True
```

Question 5: Resolving Conflicting Security Constraints

An access code must satisfy the following linear congruence:

$$14x \equiv 30 \pmod{100}$$

a) Determine whether the congruence has a solution.

Solution:

The congruence has solution if and only if $\gcd(14, 100) \mid 30$

here, $\gcd(14, 100) = 2$

and $30 = 2 * 15 + 0$

which is $2 \mid 30$ or $\gcd(14, 100) \mid 30$

Hence, the congruence has solution.

b) Find all possible solutions for x.

Solution:

$$14 * x \equiv 30(mod 100)$$

$$\text{or, } 7 * x \equiv 15(mod 100)$$

$$\text{or, } x \equiv 7^{-1} * 15(mod 100)$$

To calculate modular inverse of 7 modulo 100

we write,

$$7 * y \equiv 1(mod 100)$$

where y is the modular inverse

now,

$$100 = 14 * 7 + 2 \dots\dots(1)$$

$7 = 3 * 2 + 1 \dots\dots(2)$
 $2 = 2 * 1 + 0 \dots\dots(3)$
 from eq (1),
 $2 = 100 - (14 * 7)$
 from eq (2),
 $1 = 7 - 3 * 2$
 or, $1 = 7 - 3 * (100 - 14 * 7)$
 or, $1 = 43 * 7 - 3 * 100$
 or, $7 * 43 \equiv 1(mod100)$
 Hence, $7^{-1} \equiv 43(mod100)$
 so, $x \equiv 15 * 43(mod100)$
 or, $x \equiv 45(mod50)$
 Hence, $x = 45 + 50 * k$
 for $k = 0, 1$ we get $x=45, 95$

c) Verify each solution using Python.

Solution:

Code:

```

def check(a,n,inv):
    return ((a*inv)%n==30)
def main():
    a , n, inv1, inv2 = 14, 100 , 45, 95
    print(f"The first solution is {inv1}: {check(a,n, inv1)}")
    print(f"The second solution is {inv2}: {check(a,n, inv2)}")
if __name__=='__main__':
    main()

```

Output:

```

.. The first solution is 45: True
   The second solution is 95: True

```

d) Explain why multiple valid access codes are possible in this case.

Solution:

Multiple valid access codes are possible in this case. Because we know, when $\gcd(a, n) = d > 1$
 And $d \mid b$, there exists exactly d distinct solution modulo n . In this case,
 $14x \equiv 30(mod100)$
 $\gcd(14, 100) = 2 > 1$, and $2 \mid 30$. So we get 2 distinct solution : 45, 95